Whitebox AES : implementation and attacks

26 février 2026

Table des matières

1 Introduction **3**

2 Définition de Whitebox et Blackbox **3**
2.1 Modèle Blackbox . . . 3
2.1.1 Définition . . . 3
2.1.2 But . . . 3
2.1.3 Fonctionnement . . . 3
2.2 Modèle Whitebox . . . 4
2.2.1 Définition . . . 4
2.2.2 But . . . 4
2.2.3 Fonctionnement . . . 4

3 Implémentation **5**
3.1 Présentation générale de l'AES . . . 5
3.2 Structure de notre implémentation . . . 5
3.3 La fonction KeyExpansion . . . 8
3.4 La fonction `gf_mul` . . . 12
3.5 Les fonctions de diffusion et de confusion de chiffrement . . . 12
3.5.1 AddRoundKey . . . 13
3.5.2 ShiftRows . . . 13
3.5.3 SubBytes . . . 14
3.5.4 MixColumns . . . 15
3.5.5 Cipher . . . 16
3.6 Les fonctions de diffusion et de confusion de déchiffrement . . . 18
3.6.1 InvShiftRows . . . 18
3.6.2 InvSubBytes . . . 19
3.6.3 InvMixColumns . . . 20
3.6.4 InvCipher . . . 21
3.7 Implémentation des Lookup tables . . . 21
3.7.1 T-boxes . . . 22
3.7.2 Tables Ty_i . . . 23
3.7.3 Tables XOR . . . 24
3.7.4 Table composée . . . 24
3.7.5 Utilisation des tables . . . 25
3.8 Compilation et génération des tables . . . 27

4 Protection de l'implémentation **28**
4.1 Encodage/Masquage 28
4.1.1 Définition 28
4.1.2 Masquage des tables Ty_i et des T-boxes 28
4.1.3 Masquage des tables composés 29
4.2 Matrices de bijections 30
4.2.1 Nécessité de ces matrices 30
4.2.2 Définition et principe 30

5 Attaque BGE **34**
5.1 Structure des tables composées 35
5.2 Principe de l'attaque 35
5.3 Application sur notre implémentation 37

6 Conclusion **39**

1 Introduction

La Whitebox AES est un schéma de chiffrement par bloc proposé par Chow, Eisen, Johnson et van Oorshot en 2002. Ce schéma repose sur une alternance d'opérations de confusion et de diffusion. Il a été conçu comme une alternative plus sécurisée au DES, qui a été remplacé après un concours de chiffrement organisé entre 1997 et 2001. Ce chiffrement est largement utilisé dans des protocoles comme SSL/TLS ou dans les systèmes de stockage des données. Ce chiffrement est dit symétrique car la même clé est utilisée pour chiffrer et déchiffrer.

Pour notre implémentation, nous avons utilisé le document du NIST 197 [2] de l'AES ainsi que le document 'A Tutorial on Whitebox AES' [1] comme références principales.

Ce rapport est organisé comme suit. Chaque notion est détaillée dans sa section respective :

- Tout d'abord, nous introduisons les concepts de whitebox et de blackbox (section 2).
- Ensuite, nous présentons notre implémentation de l'AES (section 3).
- Par la suite, nous discutons des protections mises en place pour sécuriser notre implémentation (section 4).
- Enfin, nous présentons l'attaque BGE puis une attaque sur notre implémentation whitebox, bien qu'elle soit protégée (section 5).

2 Définition de Whitebox et Blackbox

Tout d'abord, introduisons le concept de whitebox et de blackbox. Une whitebox (boîte blanche) se réfère à un système ou un processus où l'intérieur est totalement connu et accessible. Alors qu'une blackbox (boîte noire), au contraire, est un système ou un processus dont l'intérieur est inconnu ou inaccessible.

2.1 Modèle Blackbox

2.1.1 Définition

Le modèle *blackbox* est le modèle classique en cryptographie. Dans ce cadre, l'attaquant n'a accès qu'aux entrées et sorties de l'algorithme cryptographique. Il ne connaît ni le code interne, ni la structure du chiffrement, ni la clé secrète utilisée. C'est ce modèle qui est utilisé dans les systèmes classiques comme TLS, les cartes à puces ou les communications sécurisées sur Internet.

2.1.2 But

Le but du modèle blackbox est de garantir la sécurité de l'algorithme et de la clé tant que l'algorithme est utilisé comme une boîte noire. Cela signifie que l'attaquant ne doit pas être capable de deviner ou retrouver la clé secrète uniquement en observant les résultats produits pour différentes entrées.

2.1.3 Fonctionnement

Dans ce modèle :

— L'utilisateur envoie un message clair à un algorithme de chiffrement ;
— Le chiffrement est réalisé à l'aide d'une clé secrète stockée dans un espace sécurisé ;
— L'attaquant ne voit que le message chiffré (la sortie) sans avoir accès aux opérations internes.

Les algorithmes sont généralement conçus pour résister à des attaques dites *chosen plaintext* (CPA) ou *chosen ciphertext* (CCA), dans lesquelles l'attaquant peut manipuler les entrées mais pas le fonctionnement interne.

2.2 Modèle Whitebox

2.2.1 Définition

Le modèle *whitebox* est l'opposé du modèle blackbox. Ici, on suppose que l'attaquant dispose d'un accès complet au programme, à son code source, à sa mémoire, ainsi qu'à son environnement d'exécution. Contrairement au modèle blackbox, l'attaquant peut observer et modifier tout ce qui se passe durant l'exécution du chiffrement.

Ce modèle est particulièrement pertinent pour les applications exécutées dans des environnements non fiables, comme les logiciels de protection de contenus (DRM), les applications mobiles, ou tout système où la clé est embarquée dans le code.

2.2.2 But

L'objectif principal d'une implémentation whitebox est de rendre extrêmement difficile, voire impossible, l'extraction de la clé secrète, même lorsque l'attaquant a un accès total au logiciel. Elle vise à assurer que la confidentialité de la clé est maintenue, même si le code est analysé, modifié ou exécuté dans un environnement contrôlé par un attaquant.

2.2.3 Fonctionnement

Une implémentation whitebox repose sur plusieurs mécanismes pour masquer la clé et la logique interne du chiffrement :
— des **transformations algébriques** comme les substitutions, permutations et opérations logiques sont combinées de manière complexe ;
— des **encodages internes** sont appliqués entre les étapes pour masquer les valeurs intermédiaires ;
— des **tables de recherche** (*lookup tables*) sont pré-calculées : elles combinent les opérations cryptographiques et l'encodage, de manière à ce que la clé ne soit jamais manipulée directement.

L'objectif est d'obfusquer complètement la structure interne de l'algorithme et d'empêcher toute tentative d'analyse ou d'extraction de la clé, même par des moyens puissants comme l'analyse différentielle computationnelle (DCA), l'analyse mémoire, ou la rétro-ingénierie.

3 Implémentation

L'AES (Advanced Encryption Standard) est un algorithme de chiffrement symétrique, ce qui signifie qu'il utilise la même clé secrète pour le chiffrement et le déchiffrement des données. Adopté comme standard par le NIST en 2001, l'AES est aujourd'hui largement utilisé dans de nombreux protocoles de sécurité (TLS, VPN, disques chiffrés, etc.).

3.1 Présentation générale de l'AES

L'algorithme AES fonctionne sur des blocs de 128 bits (16 octets) et supporte des longueurs de clé de 128, 192 ou 256 bits. Le chiffrement AES repose sur un nombre fixe de tours (ou *rounds*) :

— 10 tours pour AES-128,
— 12 tours pour AES-192,
— 14 tours pour AES-256.

Chaque tour applique une série d'opérations sur un bloc de données organisé sous forme d'une matrice 4x4 d'octets appelée *state*. Les opérations principales sont :

— **SubBytes** : une substitution non linéaire appliquée à chaque octet via une S-box (boîte de substitution) ;
— **ShiftRows** : une permutation des lignes de la matrice ;
— **MixColumns** : un mélange des colonnes utilisant des multiplications dans le corps fini $\mathbb{GF}(2^8)$;
— **AddRoundKey** : un XOR entre le *state* et une sous-clé dérivée de la clé principale.

L'ensemble des sous-clés est généré à partir de la clé principale à l'aide d'un algorithme appelé `KeyExpansion`, qui dérive une clé de ronde pour chaque tour.

3.2 Structure de notre implémentation

Cette partie se divise en deux sections : une première implémentation non sécurisée de l'AES en utilisant les fonctions classiques et standards définies par la spécification, puis une seconde implémentation différente, basée sur le principe du whitebox.

Dans cette deuxième version, l'algorithme est transformé de manière à masquer la structure interne du chiffrement. Elle repose principalement sur l'utilisation de trois types de tables de substitution :

— Les **T-boxes**, qui combinent plusieurs opérations de l'AES (SubBytes et AddRoundKey) en une seule table de consultation ;
— Les tables Ty_i, qui remplace l'opération MixColumns ;
— Les **tables XOR**, qui permettent de gérer l'opération XOR sur des données pré-encodées.

Avant de détailler la construction de ces tables et leur utilisation dans l'implémentation whitebox, nous présentons d'abord les fonctions de base utilisées dans les deux versions : la gestion des blocs, la manipulation de la clé, ainsi que les fonctions essentielles comme `KeyExpansion` pour la dérivation des sous-clés, et `gf_mul` pour les multiplications dans le corps fini $\mathbb{GF}(2^8)$. Pour ce projet, nous avons utilisé la structure suivante :

```
struct block_t {
uint8_t **block;
};

...
typedef struct block_t block_t;
...
```

Cette structure nous permet de manipuler tous les bits de notre message via un pointeur. Ce pointeur pointe vers une matrice carrée où chaque case de cette matrice est de type `uint8_t`.

Pour un message de 128 bits, il suffit d'allouer une matrice de taille 4×4.

La structure block_t représente un message ou une clé déclaré de la façon suivante :

```
static const uint8_t input_data[16] = {0x32, 0x43, 0xf6, 0xa8, 0x88,
↪  0x5a, 0x30, 0x8d, 0x31, 0x31, 0x98, 0xa2, 0xe0, 0x37, 0x07, 0x34};
// For plaintext / ciphertext
block_t *input = block_alloc(Nb);
convert_to_block(input_data, input);
// For key
block_t *key = block_alloc(Nb);
convert_to_block(key_data, key);
```

Ici, `input` représente un message d'entrée et ses valeurs sont initialisées grâce à la fonction `convert_to_block` et `key` représente la clé d'entrée et ses valeurs sont initialisées grâce à la même fonction. On peut le représenter ainsi :

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

où $s_{i,j}$ est un élément de 8 bits du state à la position (i, j). Par exemple, pour le message 3243f6a8885a308d313198a2e0370734, on a :

$$\begin{bmatrix} 32 & 88 & 31 & e0 \\ 43 & 5a & 31 & 37 \\ f6 & 30 & 98 & 07 \\ a8 & 8d & a2 & 34 \end{bmatrix}$$

Il en va de même pour les clés. La fonction `convert_to_block` remplit une structure `block_t` en copiant les octets du tableau input dans ses cases, en parcourant la structure colonne par colonne.

```
void convert_to_block(uint8_t *input, block_t *block) {
  if (!input || !block)
    return;

  int k = 0;
  for (size_t i = 0; i < Nb; i++) {
```

```
    for (size_t j = 0; j < Nb; j++) {
      block_set(block, input[k], j, i);
      k++;
    }
  }
}
```

Pour [32,43,f6,a8,88,5a,30,8d,31,31,98,a2,e0,37,07,34], le résultat de l'exécution de la fonction est :

$$[32, 43, f6, a8, 88, 5a, 30, 8d, 31, 31, 98, a2, e0, 37, 07, 34] \rightarrow \begin{bmatrix} 32 & 88 & 31 & e0 \\ 43 & 5a & 31 & 37 \\ f6 & 30 & 98 & 07 \\ a8 & 8d & a2 & 34 \end{bmatrix}$$

Ensuite, nous avons les fonctions `block_alloc` et `block_free`, qui allouent et désallouent dynamiquement l'espace mémoire pour le pointeur concerné.

```
block_t *block_alloc(const size_t size) {
  block_t *block = malloc(sizeof(block_t));
  if (!block)
    return NULL;

  block->block = calloc(size, sizeof(uint8_t *));
  if (!block->block) {
    free(block);
    return NULL;
  }

  for (size_t i = 0; i < size; i++) {
    block->block[i] = calloc(size, sizeof(uint8_t));
    if (!block->block[i]) {
      for (size_t j = 0; j < i; j++) {
        free(block->block[j]);
      }
      free(block->block);
      free(block);
      return NULL;
    }
  }
  return block;
}

void block_free(block_t *block) {
  if (!block)
    return;

  for (size_t i = 0; i < Nb; i++) {
    free(block->block[i]);
  }
  free(block->block);
  free(block);
}
```

Les fonctions `block_get` et `block_set` sont définies pour lire une case du bloc et écrire une valeur dans une case du bloc.

```
uint8_t block_get(const block_t *block, const size_t row, const size_t
↪  column) {
  if (!block || row >= Nb || column >= Nb)
    return 0;

  return block->block[row][column];
}

void block_set(const block_t *block, uint8_t value, const size_t row,
               const size_t column) {
  if (!block)
    return;
  block->block[row][column] = value;
}
```

3.3 La fonction KeyExpansion

La fonction KeyExpansion est une étape essentielle de l'algorithme AES, qui permet de dériver plusieurs clés de ronde à partir de la clé principale. Plutôt que d'utiliser cette clé unique à chaque étape du chiffrement, on utilise une clé différente pour chaque tour afin d'augmenter la sécurité du processus. La fonction KeyExpansion génère donc un ensemble complet de sous-clés pour garantir que chaque clé de tour soit unique et difficile à deviner.
La clé d'origine est divisée en mots de 4 octets chacun. Le nombre de mots dépend de la taille de la clé : 4 mots pour AES-128, 6 pour AES-192 et 8 pour AES-256.

Voici un exemple de clé initial :

$$key = [2b, 7e, 15, 16, 28, ae, d2, a6, ab, f7, 15, 88, 09, cf, 4f, 3c]$$

Quatre mots générés à partir de cette clé sont :

$$W0 = [2b, 7e, 15, 16]$$
$$W1 = [28, ae, d2, a6]$$
$$W2 = [ab, f7, 15, 88]$$
$$W3 = [09, cf, 4f, 3c]$$

Ensuite, de nouveaux mots sont générés à l'aide d'opérations mathématiques, jusqu'à obtenir un total de $Nb \times (Nr + 1)$ mots, soit 44 mots dans le cas de AES-128, où Nb est le nombre de colonnes de l'état (toujours égal à 4 en AES) et Nr est le nombre de tours (10, 12 ou 14 selon la taille de la clé). Pour créer chaque nouveau mot, on applique généralement une opération XOR entre le mot précédent et un mot situé Nk positions avant.

$$w_i = w_{i-1} \oplus w_{i-Nk}$$

Ici, Nk correspond au nombre de mots de 32 bits composant la clé principale : 4, 6 ou 8 selon que la clé fait 128, 192 ou 256 bits.
À certains moments (lorsque l'indice du mot est un multiple de Nk), on transforme le mot précédent avec deux fonctions RotWord et SubWord. Une constante spéciale appelée Rcon[i] est également utilisée à ces moments-là pour renforcer la sécurité.

Dans notre implémentation, nous avons utilisé un tableau de stockage appelé `round_key` dans lequel nous stockons 44 mots : les 4 mots de la clé initiale, suivis de 40 mots générés pour les 10 clés de ronde.

```
void key_expansion(uint8_t round_key[Nb * (Nr + 1)][Nb], block_t *key)
↪ {
 if (!round_key || !key)
   return;

 //* The first round_key is the key itself
 for (size_t i = 0; i < Nb; i++) {
   for (size_t j = 0; j < Nb; j++) {
     round_key[i][j] = key->block[i][j];
   }
 }

 uint8_t tmp[Nb];
 for (size_t i = Nk; i < Nb * (Nr + 1); i++)
 {
   for (size_t j = 0; j < Nb; j++)
   {
     tmp[j] = round_key[i - 1][j];
   }

   if (i % Nk == 0)
   {
     rot_word(tmp);
     sub_word(tmp);
     tmp[0] ^= rcon[(i / Nk)];
   }

   for (size_t j = 0; j < Nb; j++) {
     round_key[i][j] = round_key[i - Nk][j] ^ tmp[j];
   }
 }
}
```

La fonction KeyExpansion appelle deux fonctions : RotWord et SubWord. Ces deux fonctions sont utilisées pour introduire respectivement de la diffusion (par rotation des octets) et de la non-linéarité (par substitution via les S_box), renforçant ainsi la sécurité du chiffrement.

La fonction RotWord effectue un décalage circulaire des octets au sein du mot. Voici un exemple du résultat produit par la fonction :

$$[a0, a1, a2, a3] \rightarrow [a1, a2, a3, a0]$$

La fonction SubWord applique la S-box (table de substitution) à chaque octet du mot d'entrée.
Si on a un mot de 4 octets alors chaque octet est remplacé par sa correspondance dans la S-box AES. En reprenant le résultat de la fonction RotWord, on obtient :

$$[a1, a2, a3, a0] \rightarrow [S_box(a1), S_box(a2), S_box(a3), S_box(a0)]$$

Après l'application des fonctions RotWord et SubWord, une troisième transformation est effectuée : l'ajout d'une constante de ronde, appelée Rcon. Cette

constante provient d'une table, Rcon, qui contient les puissances successives de l'élément x dans le corps fini $\mathbb{GF}(2^8)$. Ce champ, utilisé dans l'algorithme AES, permet d'effectuer des opérations binaires de manière efficace et sécurisée. Chaque valeur de Rcon correspond à une puissance de x dans $\mathbb{GF}(2^8)$, calculée avec une réduction modulo un polynôme irréductible :

$$x^8 + x^4 + x^3 + x + 1$$

Ce polynôme garantit que le résultat des opérations reste contenu dans un seul octet, ce qui est essentiel pour le bon fonctionnement du chiffrement AES.

```
/* powers of x in the field GF(2^8) */
static const uint8_t rcon[11] = {0x00, 0x01, 0x02, 0x04, 0x08, 0x10,
                                 0x20, 0x40, 0x80, 0x1b, 0x36};

 static void rot_word(uint8_t *word) {
  if (!word)
    return;

  uint8_t tmp = word[0];
  for (size_t i = 0; i < Nb - 1; i++)
    word[i] = word[i + 1];

  word[Nb - 1] = tmp;
}

/* applies the S-box */
static void sub_word(uint8_t *word) {
  if (!word)
    return;

  for (size_t i = 0; i < Nb; i++) {
    word[i] = s_box[word[i]];
  }
}
```

La fonction `get_key`, utilisée plus tard dans la fonction cipher, permet d'extraire les mots de la clé de tour correspondant au numéro du round.
Au sein de la fonction `get_key`, on appelle la fonction `rotate_block` qui inverse les lignes et les colonnes. Voici un exemple concret de ce que fait `rotate_block` :

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \rightarrow \begin{bmatrix} s_{0,0} & s_{1,0} & s_{2,0} & s_{3,0} \\ s_{0,1} & s_{1,1} & s_{2,1} & s_{3,1} \\ s_{0,2} & s_{1,2} & s_{2,2} & s_{3,2} \\ s_{0,3} & s_{1,3} & s_{2,3} & s_{3,3} \end{bmatrix}$$

En effet, après l'extraction des mots correspondant à chaque clé de tour, chaque mot est censé être une colonne de la clé de tour. C'est pourquoi on commence par extraire quatre mots, puis on utilise chaque mot de 32 bits pour constituer une colonne de la clé de tour. Cette étape est cruciale pour organiser la clé de tour dans le bon ordre, conformément aux exigences de l'algorithme de chiffrement.

Par exemple, si l'on a les quatre mots ci-dessous, correspondant aux mots de la première clé de tour :

$$\ldots$$
$$W4 = [a0, fa, fe, 17]$$
$$W5 = [88, 54, 2c, b1]$$
$$W6 = [23, a3, 39, 39]$$
$$W7 = [2a, 6c, 76, 05]$$
$$\ldots$$

La fonction `get_key` nous génère :

$$\begin{bmatrix} a0 & fa & fe & 17 \\ 88 & 54 & 2c & b1 \\ 23 & a3 & 39 & 39 \\ 2a & 6c & 76 & 05 \end{bmatrix}$$

Et donc, après l'appel de la fonction `rotate_block` dans la fonction `get_key`, on obtient la clé de tour suivante :

$$\begin{bmatrix} a0 & fa & fe & 17 \\ 88 & 54 & 2c & b1 \\ 23 & a3 & 39 & 39 \\ 2a & 6c & 76 & 05 \end{bmatrix} \rightarrow \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$$

Voici notre implémentation de ces deux fonctions :

```
static void rotate_block(block_t *block) {
  if (!block)
    return;

  block_t *tmp = block_alloc(Nb);
  block_copy(block, tmp);
  for (size_t i = 0; i < Nb; i++) {
    for (size_t j = 0; j < Nb; j++) {
      block->block[i][j] = tmp->block[j][i];
    }
  }
}

static void get_key(uint8_t round_key[Nb * (Nr + 1)][Nb], block_t *key,
                    size_t round) {
  if (!round_key || !key)
    return;

  size_t start = (round * 4);
  size_t k = 0;
  for (size_t i = start; i < start + 4; i++) {
    for (size_t j = 0; j < Nb; j++) {
      key->block[k][j] = round_key[i][j];
    }
    k++;
```

```
  }
  rotate_block(key);
}
```

On peut générer les clés de tours comme suit :

```
uint8_t key_data[16] = {0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6,
↪  0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c};
uint8_t round_key[Nb * (Nr + 1)][Nb];
block_t *key = block_alloc (Nb);
convert_to_block (key_data, key);
key_expansion (round_key, key);
```

Ici, `round_key` contient toutes les clés de tours ainsi que la clé initiale, qui est, dans cet exemple, 2b7e151628aed2a6abf7158809cf4f3c.

3.4 La fonction `gf_mul`

La fonction `gf_mul` applique une multiplication de deux éléments dans le corps de Galois $\mathrm{GF}(2^8)$. Dans cette implémentation, chaque valeur est considérée comme un polynôme de degré inférieur ou égal à 7 sur $\mathbb{F}_2$, où le bit de poids faible correspond au terme constant et le bit de poids fort au terme de degré 7. La multiplication est réalisée en appliquant l'algorithme de multiplication et en réduisant modulo le polynôme irréductible :

$$x^8 + x^4 + x^3 + x + 1 = 0x11B.$$

A chaque étape, la variable `a` est multipliée par `x` (décalage à gauche). Si `a` dépasse 8 bits (bit de poids fort à 1), une réduction modulo le polynôme irréductible est effectuée.

```
#define POLYNOMIAL 0x1B
uint8_t gf_mul(uint8_t a, uint8_t b) {
  uint8_t result = 0;
  for (; b; b >>= 1) {
    if (b & 1) {
      result ^= a;
    }
    if (a & 0x80) {
      a = (a << 1) ^ POLYNOMIAL;
    } else {
      a <<= 1;
    }
  }
  return result;
}
```

3.5 Les fonctions de diffusion et de confusion de chiffrement

Tout d'abord, expliquons les principes de diffusion/confusion. Le principe de diffusion consiste à étaler ou disperser l'information d'une manière qui rend la relation entre le texte en clair et le texte chiffré aussi complexe que possible.

En résumé, le but est de propager les changements d'un bit du texte clair sur plusieurs bits du texte chiffré. On compte notamment comme exemples d'opérations des permutation de bits ou d'octets ou encore les mélanges linéaires de bits. Puis le principe de confusion vise à rendre la relation entre le texte en clair, la clé et le texte chiffré aussi complexe et non linéaire que possible. En résumé, cela consiste à cacher la relation entre la clé et le texte chiffré. Des exemples d'opérations sont la substitution non-linéaire ou encore des opérations avec la clé.

3.5.1 AddRoundKey

La fonction AddRoundKey est une opération de confusion qui modifie l'état du message, case en case, en appliquant un XOR avec la clé de tour correspondante. On a donc :

$$s_{i,j} \oplus w_{i,j} = s'_{i,j} \text{ pour i=0...3 et j=0...3}$$

où $s_{i,j}$ sont les éléments de l'état du message, $w_{i,j}$ sont les éléments de la clé de tour et $s'_{i,j}$ est le résultat final. La fonction est écrite comme suit :

```
void add_round_key(block_t *block, block_t *round_key) {
  if (!block || !round_key) {
    return;
  }

  for (size_t i = 0; i < block->size; i++) {
    for (size_t j = 0; j < block->size; j++) {
      block->block[i][j] ^= round_key->block[i][j];
    }
  }
}
```

3.5.2 ShiftRows

La fonction ShiftRows est une opération de diffusion qui effectue un décalage circulaire des trois dernières lignes de l'état du message vers la gauche. La première ligne reste inchangée, tandis que les deuxième, troisième et quatrième lignes sont décalées respectivement d'une, deux et trois positions vers la gauche, ce qui peut être représenté comme suit :

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \rightarrow \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

Pour ce faire, nous avons implémenté une fonction auxiliaire `shift_left` qui effectue une rotation circulaire sur une ligne vers la gauche. Ensuite, selon du numéro de la ligne, cette fonction est appelée pour appliquer le décalage circulaire approprié.

```c
static void shift_left(block_t *block, size_t row) {
  if (!block)
  return;

  uint8_t tmp;
  tmp = block->block[row][0];
  for (size_t column = 0; column < block->size; column++) {
    if (column == block->size - 1)
      block->block[row][column] = tmp;
    else
      block->block[row][column] = block->block[row][column + 1];
    }
  }

void shift_rows(block_t *block) {
  if (!block)
    return;

  /* shift value depends on the row number */
  for (size_t row = 0; row < block->size; row++) {
    for (size_t j = 0; j < row; j++) {
      shift_left(block, row);
    }
  }
}
```

3.5.3 SubBytes

La fonction `SubBytes` est une fonction de substitution qui remplace chaque octet du message par une valeur provenant de la table de substitution, appelée `S_box`. Chaque octet de l'état est remplacé par sa valeur correspondante dans la `S_box` afin d'introduire de la confusion dans le chiffrement. Cette substitution est réalisée à l'aide d'une table pré-calculée qui associe chaque octet à une autre valeur.

L'opération de substitution devient alors :

$$s'_{i,j} = \text{s_box}(s_{i,j})$$

Où $s'_{i,j}$ est l'octet substitué à la position (i, j) dans l'état et $s_{i,j}$ est l'octet original à la même position. La table `S_box` est construite à partir de l'inverse multiplicatif dans le champ fini $GF(2^8)$, suivi d'une transformation affine.

La S-Box de l'AES est construite en deux étapes principales :
1. Inverse multiplicatif dans $GF(2^8)$: Chaque octet x de $GF(2^8)$ (représenté comme un entier de 8 bits) est remplacé par son inverse multiplicatif dans ce champ, sauf pour l'octet 0x00, qui est une exception. L'inverse multiplicatif x^{-1} est calculé selon les règles de l'algèbre de Galois, où l'inverse de x satisfait :

$$x \cdot x^{-1} = 1 \quad \text{modulo} \quad x^8 + x^4 + x^3 + x + 1$$

2. Transformation affine : Une fois l'inverse calculé, une transformation affine est appliquée à l'inverse multiplicatif. La transformation affine est définie par la matrice suivante :

$$x' = A \cdot x + c$$

où A est une matrice 8×8 sur $GF(2)$, et c est un vecteur constant de 8 bits. Cela permet de mélanger davantage les bits et d'assurer que la substitution ne puisse pas être facilement inversée sans connaître la clé.

Une fois la `S_box` calculée et stockée, l'étape de substitution consiste à remplacer chaque octet de l'état par sa valeur correspondante dans la table `S_box`. Cette opération est répétée pour chaque octet de l'état (un état étant un tableau de 4x4 octets).

La fonction SubBytes peut être définie comme suit :

```
void sub_bytes(block_t *block) {
  if (!block) {
    return;
  }

  for (size_t i = 0; i < Nb; i++) {
    for (size_t j = 0; j < Nb; j++) {
      uint8_t byte = block_get(block, i, j);  // Récupère l'octet de
      ↪  l'état
      uint8_t substituted = s_box[byte];      // Applique la
      ↪  substitution via la S-Box
      block_set(block, substituted, i, j);    // Remplace l'octet dans
      ↪  l'état
    }
  }
}
```

Dans cette fonction :

- La fonction `block_get` récupère l'octet à la position (i, j) de l'état.
- La table `S_box` est utilisée pour substituer l'octet récupéré.
- La fonction `block_set` remplace l'octet original par l'octet substitué dans l'état.

Ce processus introduit de la confusion dans le chiffrement en rendant les relations entre les octets de l'état et la clé secrète beaucoup plus complexes.

L'opération SubBytes est effectuée à chaque tour du chiffrement AES et est fondamentale pour sécuriser le processus de transformation en rendant difficile l'analyse des relations algébriques dans l'état. L'utilisation de la S-Box fait partie de ce qu'on appelle la "confusion" dans la théorie du chiffrement, un des deux principes fondamentaux (avec la diffusion) pour rendre un chiffrement résistant aux attaques par cryptanalyse

3.5.4 MixColumns

La fonction MixColumns est une fonction de diffusion qui utilise une matrice MC à coefficients et multiplie la matrice avec le message en paramètre dans $\mathrm{GF}(2^8)$. Chaque colonne est considérée comme un polynôme dans $\mathrm{GF}(2^8)$. Concrètement, pour chaque colonne de l'état, la matrice MC (qui est une matrice 4×4 fixe)

est multipliée par la colonne correspondante du message. L'opération devient :

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

Que l'on peut représenter tout simplement :

$$\begin{aligned} s'_{0,c} &= (02 \times s_{0,c}) \oplus (03 \times s_{1,c}) \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= s_{0,c} \oplus (02 \times s_{1,c}) \oplus (03 \times s_{2,c}) \oplus s_{3,c} \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (02 \times s_{2,c}) \oplus (03 \times s_{3,c}) \\ s'_{3,c} &= (03 \times s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (02 \times s_{3,c}) \end{aligned}$$

La fonction est écrite ainsi :

```c
void mix_columns(block_t *block) {
  if (!block)
    return;

  for (size_t i = 0; i < block->size; i++) {
    uint8_t s0 = block->block[0][i];
    uint8_t s1 = block->block[1][i];
    uint8_t s2 = block->block[2][i];
    uint8_t s3 = block->block[3][i];

    uint8_t s0x2 = gf_mul(s0, 0x02);
    uint8_t s1x2 = gf_mul(s1, 0x02);
    uint8_t s2x2 = gf_mul(s2, 0x02);
    uint8_t s3x2 = gf_mul(s3, 0x02);

    uint8_t s0x3 = gf_mul(s0, 0x03);
    uint8_t s1x3 = gf_mul(s1, 0x03);
    uint8_t s2x3 = gf_mul(s2, 0x03);
    uint8_t s3x3 = gf_mul(s3, 0x03);

    block->block[0][i] = s0x2 ^ s1x3 ^ s2 ^ s3;
    block->block[1][i] = s0 ^ s1x2 ^ s2x3 ^ s3;
    block->block[2][i] = s0 ^ s1 ^ s2x2 ^ s3x3;
    block->block[3][i] = s0x3 ^ s1 ^ s2 ^ s3x2;
  }
}
```

3.5.5 Cipher

La fonction cipher fait le chiffrement du message en appliquant successivement les transformations SubBytes, ShiftRows, MixColumns et AddRoundKey. Le tableau round_key contient 11 clés, comprenant la clé principale et 10 clés de tour. Avant de commencer les 10 tours de chiffrement, on applique un XOR entre la clé principale et le bloc du message. Ensuite, nous effectuons 9 tours de chiffrement en appelant les fonctions SubBytes, ShiftRows, MixColumns et AddRoundKey. Au dixième tour, on n'applique pas la fonction MixColumns. Cet ordre a été spécificement choisies pour respecter les propriétés cryptographiques.

- Cet ordre et ces fonctions standards apportent confusion et diffusion, deux principes clés en cryptographie selon Shannon.
- Cet implémentation permet d'implémenter le déchiffrement simplement en inversant l'ordre des fonctions par leurs équivalents inverses (voir section 3.6.4).

```c
  void cipher(block_t *input, uint8_t round_key[Nb * (Nr + 1)][Nb]) {
  if (!input || !round_key)
    return;

  block_t *key = block_alloc(Nb);
  get_key(round_key, key, 0);
  add_round_key(input, key);
  block_free(key);
  for (size_t round = 1; round < Nr; round++) {
    sub_bytes(input);
    shift_rows(input);
```

```
    mix_columns(input);
    key = block_alloc(Nb);
    get_key(round_key, key, round);
    add_round_key(input, key);
    block_free(key);
  }
  sub_bytes(input);
  shift_rows(input);
  block_t *last_key = block_alloc(Nb);
  get_key(round_key, last_key, Nr);
  add_round_key(input, last_key);
  block_free(last_key);
}
```

Cependant, il est possible de redéfinir Cipher avec un ordre différent mais équivalent à la première implémentation. On peut constater que :

— On peut inclure dans la boucle la fonction AddRoundKey(state, k_{r-1}) et déplacer AddRoundKey(state, k_9) aprés la fin de la boucle. En commençant la boucle au round 1, la première opération est AddRoundKey(state, k_0). Cette implémentation reste donc équivalente à la première implémentation.

— AddRoundKey(state, k_{r-1}) suivi de ShiftRows(state) donne le même résultat que ShiftRows(state) et AddRoundKey(state, $\hat{k}_{r-1}$) où $\hat{k}_{r-1}$ est le résultat de ShiftRows(k_{r-1}). ShiftRows est une permutation d'octets de l'état (c'est une transformation dite linéaire). Or AddRoundKey est une simple opération de XOR, qui est également une opération linéaire sur$GF(2^8)$. Deux opérations linéaires sur $GF(2^8)$ commutent si on applique la permutation à la clé :

$$ShiftRows(state \oplus key) = ShiftRows(state) \oplus ShiftRows(key)$$

On peut donc réecrire $AddRoundKey(state, k)$ en $AddRoundKey(ShiftRows(state), ShiftRows(k))$ comme suit :

$$AddRoundKey(ShiftRows(state), \hat{k})$$

Où $\hat{k} = ShiftRows(k)$.

La nouvelle version de Cipher devient :

```
void cipher(block_t *input, uint8_t round_key[Nb * (Nr + 1)][Nb]) {
if (!input || !round_key)
  return;

block_t *key = block_alloc(Nb);
for (size_t round = 1; round < Nr; round++) {
  // ShiftRows
  shift_rows(input);
  // AddRoundKey
  get_key(round_key, key, round - 1);
  shift_rows(key);
  add_round_key(input, key);
  // SubBytes
  sub_bytes(input);
  // MixColumns
  mix_columns(input);
  block_free(key);
}
// ShiftRows
```

```
    shift_rows(input);
    // AddRoundKey
    get_key(round_key, key, Nr - 1);
    shift_rows(key);
    add_round_key(input, key);
    block_free(key);
    // SubBytes
    sub_bytes(input);
    // AddRoundKey
    block_t *last_key = block_alloc(Nb);
    get_key(round_key, last_key, Nr);
    add_round_key(input, last_key);
    block_free(last_key);
  }
```

Cette nouvelle version permet la fusion de AddRoundKey et SubBytes (suivi d'un ShiftRows sur la clé), que l'on implémentera dans des tables de recherche dans la section 3.7.1.

3.6 Les fonctions de diffusion et de confusion de déchiffrement

3.6.1 InvShiftRows

La fonction InvShiftRows est la fonction inverse de ShiftRows.Elle effectue un décalage circulaire des trois dernières lignes de l'état du message vers la droite. La première ligne reste inchangée, tandis que les deuxième, troisième et quatrième lignes sont décalées respectivement d'une, deux et trois positions vers la droite. el'on peut représenter tout simplement :

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \rightarrow \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,3} & s_{1,0} & s_{1,1} & s_{1,2} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,1} & s_{3,2} & s_{3,3} & s_{3,0} \end{bmatrix}$$

Pour ce faire, nous avons implémenté une fonction auxiliaire `shift_right` qui effectue une rotation circulaire. Ensuite, en fonction du numéro de la ligne, cette fonction est appelée pour appliquer le décalage circulaire approprié.

```
static void shift_right(block_t *block, size_t row) {
  if (!block)
    return;

  uint8_t tmp;
  tmp = block->block[row][block->size - 1];
  for (size_t column = block->size - 1; column > 0; column--) {
    block->block[row][column] = block->block[row][column - 1];
  }

  block->block[row][0] = tmp;
}

void inverse_shift_rows(block_t *block) {
  if (!block)
    return;
```

```
  /* shift value depends on the row number */
  for (size_t row = 0; row < block->size; row++) {
    for (size_t j = 0; j < row; j++)
      shift_right(block, row);
  }
}
```

3.6.2 InvSubBytes

La fonction `InvSubBytes` est la fonction inverse de `SubBytes`. Le principe est similaire, sauf que cette fois-ci, chaque octet du bloc est remplacé par sa valeur correspondante dans la table de substitution inverse `inv_s_box`. Cette opération est utilisée dans le processus de décryption, pour inverser l'effet de la substitution effectuée lors du chiffrement.

Concrètement, pour chaque octet du bloc, la fonction cherche sa valeur dans la `inv_s_box` et remplace l'octet original par l'octet substitué inverse. L'opération devient :

$$s'_{i,j} = \text{inv_s_box}(s_{i,j})$$

où $s_{i,j}$ est l'octet original et $s'_{i,j}$ est l'octet après substitution inverse. L'inverse de la substitution est nécessaire pour récupérer l'état initial du message avant le chiffrement, permettant ainsi de retrouver le message original.

La table `inv_s_box` est construite de manière similaire à la `S_box`, mais en utilisant l'inverse multiplicatif dans le champ fini $GF(2^8)$, suivi d'une transformation affine inverse. Cela permet de "défaire" l'effet de la substitution appliquée lors du chiffrement.

1. Inverse multiplicatif dans $GF(2^8)$: Comme pour la S-Box, chaque octet x de $GF(2^8)$ est remplacé par son inverse multiplicatif dans le champ $GF(2^8)$, sauf pour l'octet 0x00, qui n'a pas d'inverse. L'inverse est calculé de manière similaire au processus de la S-Box, c'est-à-dire en résolvant l'équation :

$$x \cdot x^{-1} = 1 \quad \text{modulo} \quad x^8 + x^4 + x^3 + x + 1$$

2. Transformation affine inverse : Une fois l'inverse multiplicatif trouvé, une transformation affine inverse est appliquée. Cette transformation affine inverse est définie par :

$$x' = A^{-1} \cdot x + c^{-1}$$

où A^{-1} est l'inverse de la matrice A utilisée dans la transformation affine de la S-Box, et c^{-1} est l'inverse du vecteur constant c. Cette opération inverse la transformation affine utilisée dans la création de la S-Box, permettant ainsi de récupérer l'état original.

Une fois la `inv_s_box` calculée et stockée, l'étape de substitution inverse consiste à remplacer chaque octet de l'état par sa valeur correspondante dans la table `inv_s_box`. Cette opération est effectuée pour chaque octet de l'état (un état étant un tableau de 4x4 octets).

La fonction `InvSubBytes` en C est définie comme suit :

```
void inverse_sub_bytes(block_t *block) {
  if (!block) {
    return;
  }

  for (size_t i = 0; i < Nb; i++) {
    for (size_t j = 0; j < Nb; j++) {
      uint8_t byte = block_get(block, i, j);  // Récupère l'octet de
      ↪  l'état
```

```
      uint8_t substituted = inv_s_box[byte];  // Applique la
      ↪  substitution inverse via la inv_s_box
      block_set(block, substituted, i, j);    // Remplace l'octet dans
      ↪  l'état
    }
  }
}
```

Dans cette fonction :

— La fonction `block_get` récupère l'octet à la position (i, j) de l'état.
— La table `inv_s_box` est utilisée pour substituer l'octet récupéré par son inverse.
— La fonction `block_set` remplace l'octet original par l'octet substitué inverse dans l'état.

Cette étape de substitution inverse est réalisée lors de la décryption du message, où l'objectif est de restaurer l'état initial du message avant le chiffrement. La table `inv_s_box` permet ainsi d'annuler l'effet de la substitution opérée dans le processus de chiffrement et de retrouver le texte clair.

L'opération `inverse_sub_bytes` est essentielle dans le processus de décryption de l'AES, car elle est utilisée pour inverser la confusion introduite par la fonction `sub_bytes` lors du chiffrement. Grâce à la table `inv_s_box`, l'algorithme de décryption peut récupérer le message original sans perte d'information.

3.6.3 InvMixColumns

InvMixColumns est l'inverse de la fonction MixColumns. Le principe est le même sauf que la matrice MC est différente, cette opération devient :

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \cdot \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

Que l'on peut écrire comme :

$$\begin{aligned} s'_{0,c} &= (0e \times s_{0,c}) \oplus (0b \times s_{1,c}) \oplus (0d \times s_{2,c}) \oplus (09 \times s_{3,c}) \\ s'_{1,c} &= (09 \times s_{0,c}) \oplus (0e \times s_{1,c}) \oplus (0b \times s_{2,c}) \oplus (0d \times s_{3,c}) \\ s'_{2,c} &= (0d \times s_{0,c}) \oplus (09 \times s_{1,c}) \oplus (0e \times s_{2,c}) \oplus (0b \times s_{3,c}) \\ s'_{3,c} &= (0b \times s_{0,c}) \oplus (0d \times s_{1,c}) \oplus (09 \times s_{2,c}) \oplus (0e \times s_{3,c}) \end{aligned}$$

Le fonction devient :

```
void inverse_mix_columns(block_t *block) {
  if (!block)
    return;

  for (size_t i = 0; i < block->size; i++) {
    uint8_t s0 = block->block[0][i];
    uint8_t s1 = block->block[1][i];
    uint8_t s2 = block->block[2][i];
    uint8_t s3 = block->block[3][i];
    block->block[0][i] = gf_mul(s0, 0x0e) ^ gf_mul(s1, 0x0b) ^
                         gf_mul(s2, 0x0d) ^ gf_mul(s3, 0x09);
    block->block[1][i] = gf_mul(s0, 0x09) ^ gf_mul(s1, 0x0e) ^
                         gf_mul(s2, 0x0b) ^ gf_mul(s3, 0x0d);
```

```
    block->block[2][i] = gf_mul(s0, 0x0d) ^ gf_mul(s1, 0x09) ^
                         gf_mul(s2, 0x0e) ^ gf_mul(s3, 0x0b);
    block->block[3][i] = gf_mul(s0, 0x0b) ^ gf_mul(s1, 0x0d) ^
                         gf_mul(s2, 0x09) ^ gf_mul(s3, 0x0e);
  }
}
```

3.6.4 InvCipher

La fonction InvCipher déchiffre un message chiffré. Elle applique dans l'ordre InvShiftRows, InvSubBytes, InvMixColumns et AddRoundKey. En effet, on inverse tout simplement l'ordre des appels des fonctions dans InvCipher tout en préservant les propriétés cryptographiques (confusion/diffusion) selon Shannon. La gestion des clés de tour est la même que dans Cipher à la différence que l'on commence par la dernière clé.

```
void inverse_cipher(block_t *block, uint8_t round_key[Nb * (Nr +
↪  1)][Nb]) {
  if (!block || !round_key)
    return;

  block_t *key;
  key = block_alloc(Nb);
  get_key(round_key, key, Nr);
  add_round_key(block, key);
  block_free(key);
  for (size_t round = Nr - 1; round >= 1; round--) {
    inverse_shift_rows(block);
    inverse_sub_bytes(block);
    key = block_alloc(Nb);
    get_key(round_key, key, round);
    add_round_key(block, key);
    inverse_mix_columns(block);
    block_free(key);
  }
  block_t *last_key = block_alloc(Nb);
  get_key(round_key, last_key, 0);
  inverse_shift_rows(block);
  inverse_sub_bytes(block);
  add_round_key(block, last_key);
}
```

3.7 Implémentation des Lookup tables

Une lookup table (ou table de recherche) est une structure de données utilisée pour remplacer des calculs complexes par des recherches rapides en mémoire. Le principe est simple : au lieu d'effectuer un calcul à chaque fois, on précalcule les résultats possibles et on les stocke dans une table. Ainsi, il suffit d'accéder à l'entrée correspondante dans la table pour obtenir le résultat, ce qui permet d'accélérer considérablement l'exécution de l'algorithme en évitant des opérations répétitives et coûteuses en temps.

Dans le contexte de l'AES, les lookup tables sont particulièrement utiles pour les transformations comme SubBytes, MixColumns et AddRoundKey. Elles

permettent non seulement un gain de performance, mais peuvent aussi rendre l'analyse plus complexe pour un attaquant, en masquant certaines opérations internes. Elles sont également utilisées pour compliquer l'extraction de la clé par les attaquants. Trois lookup tables sont utilisées : les T-boxes, les tables Ty_i et les tables XOR.

3.7.1 T-boxes

Les T-boxes combinent les transformations AddRoundKey et SubBytes en une seule table de consultation pré-calculée. Chaque T-box associe un octet d'entrée à un octet de sortie en appliquant successivement une substitution non linéaire (via la S-box) suivie d'une opération XOR avec un octet de la clé de tour. Cette combinaison permet de réduire le nombre d'opérations effectuées en temps réel, car au lieu d'exécuter séparément la substitution et l'ajout de clé, on effectue une simple lecture en mémoire dans la T-box. Les T-boxes sont définies comme suit :

$$T_i^r(x) = S(x \oplus \hat{k}_{r-1}[i]) \text{ pour i=0...15 et r=1...9}$$
$$T_i^{10}(x) = S(x \oplus \hat{k}_9[i]) \oplus k_{10}[i] \text{ pour i=0...15}$$

où $\hat{k}_r$ est le résultat de l'application de la fonction ShiftRows sur la clé de tour k_r. Ainsi, pour chacun des 9 premiers tours de l'AES, on utilise 16 T-boxes, chacune correspondant à un octet précis de l'état interne. Cela représente donc 144 T-boxes au total pour les tours 1 à 9. En ajoutant les 16 T-boxes spécifiques au dixième tour, qui incorporent les octets des deux dernières clés de tour ($\hat{k}_9$ et k_{10}), on obtient un total de 160 T-boxes.

3.7.2 Tables Ty_i

Les tables Ty_i permettent de remplacer la fonction MixColumns. Cette fonction est utilisée du round 1 au round 9. L'état d'entrée comporte 4 colonnes, et pour chaque colonne, quatre tables Ty_i sont nécessaires pour réaliser l'opération MixColumns. Comme ces transformations sont appliquées au cours des 9 premiers rounds, cela donne un total de $4 \times 4 \times 9 = 144$ tables Ty_i.

On calcule, pour chaque valeur possible sur 8 bits, le résultat de la multiplication de cette valeur par une colonne de la matrice MC déjà définie. Ensuite, on stocke le résultat dans les tables concernées. Notons x_0, x_1, x_2, x_3 quatre valeurs de 8 bits multipliées par la matrice MC. Cette multiplication se décompose en plusieurs XORs tels que :

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 02 \\ 01 \\ 01 \\ 03 \end{bmatrix} x_0 \oplus \begin{bmatrix} 03 \\ 02 \\ 01 \\ 01 \end{bmatrix} x_1 \oplus \begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} x_2 \oplus \begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} x_3$$

Chacun des termes de la somme à droite (que l'on note y_0, y_1, y_2, y_3) dépend d'un unique octet d'entrée. Par conséquent, chaque y_i peut prendre 256 valeurs possibles. Avec y_i, nous créons les tables Ty_i définies comme suit :

$$\begin{aligned} Ty_0(x) &= x \cdot [02\ 01\ 01\ 03]^T \\ Ty_1(x) &= x \cdot [03\ 02\ 01\ 01]^T \\ Ty_2(x) &= x \cdot [01\ 03\ 02\ 01]^T \\ Ty_3(x) &= x \cdot [01\ 01\ 03\ 02]^T \end{aligned}$$

La génération des tables Ty_i peut être implémentée de la manière suivante :

```
#define TABLE_NUMBER 4
#define NB_ROUNDS 10
#define POSSIBLE_VALUES 256

uint32_t tyi_tables_data[NB_ROUNDS - 1][TABLE_NUMBER][TABLE_NUMBER][POSSIBLE_VALUES] = {
↪  0 };

static void generate_tyi_tables ()
{
  for (int r = 0; r < NB_ROUNDS - 1; r++)
  {
    for (int byte = 0; byte < TABLE_NUMBER; byte++)
    {
      for (int x = 0; x < POSSIBLE_VALUES; x++)
        {
          tyi_tables_data[r][byte][0][x]
              = (((gf_mul (2, x) << 24) | (x << 16) | (x << 8) | gf_mul (3, x)));
          tyi_tables_data[r][byte][1][x]
              = ((gf_mul (3, x) << 24) | (gf_mul (2, x) << 16) | (x << 8) | x) ;
          tyi_tables_data[r][byte][2][x]
              = ((x << 24) | (gf_mul (3, x) << 16) | (gf_mul (2, x) << 8) | x);
          tyi_tables_data[r][byte][3][x]
              = ((x << 24) | (x << 16) | (gf_mul (3, x) << 8) | gf_mul (2, x)) ;
        }
    }
  }
}
```

On utilisera par la suite cette table pour calculer le résultat de MixColumns de la manière suivante :

$$Ty_0(x_0) \oplus Ty_1(x_1) \oplus Ty_2(x_2) \oplus Ty_3(x_3)$$

3.7.3 Tables XOR

Les tables XOR permettent de remplacer l'opération XOR entre deux nibbles (entiers de 4 bits). Ces nibbles sont représentés par des valeurs allant de 0 à 15. On utilise des nibbles pour accélérer le temps de calcul de la table et lors de son utilisation. Elles sont utilisées pour l'opération XOR dans MixColumns, après l'extraction des valeurs dans les tables Ty_i.

$$\text{XOR}(x, y) = x \oplus y \text{ pour x=0...15 et y=0...15}$$

À chaque tour (de 1 à 9), douze opérations XOR sur des valeurs de 32 bits doivent être effectuées pour calculer le résultat de MixColumns.
Nous avons besoin de 864 copies de la table XOR car :

- Chaque MixColumns combine des sorties de tables Ty_i (chaque sortie fait 32 bits). Pour effectuer l'opération $\oplus$ entre deux valeurs de 32 bits, on découpe chaque valeur en 8 nibbles (de 4 bits), donc il nous faut 8 tables XOR pour une seule opération XOR de 32 bits.
- Dans un tour de l'AES, on applique MixColumns sur 4 colonnes de l'état, et chaque colonne nécessite 3 XORs pour combiner les 4 sorties de la table Ty_i. Cela donne : $4 \times 3 = 12$ XORs de 32 bits par tour.
- Chaque XOR 32 bits utilise 8 nibbles de 4 bits : $12 \times 8 = 96$ tables par tour.
- Sachant que nous avons 9 tours au total (nous n'appliquons pas MixColumn au dernier tour) : $96 \times 9 = 864$ tables.

On pourrait penser qu'une seule table XOR suffirait, mais la protection présentée dans la section 4 (Protection de l'implémentation) ne le permet pas. La génération des tables XOR est implémentée de la manière suivante :

3.7.4 Table composée

Comme on utilise les T-boxes juste avant les Ty_i, on peut remplacer ces deux tables par leur composition. Composer ces deux tables permet de réduire le temps de calcul entre les tours 1 et 9. $Ty_i(x)$ représente la multiplication de la matrice MC pour une valeur d'entrée x sur la colonne i. Les T-boxes (notées T_i^r) représente quand à elles l'opération AddRoundKey et SubBytes. On extrait tout d'abord les valeurs des T-boxes, puis des quatre colonnes de la table Ty_i. On peut donc écrire une nouvelle table $Ty_boxes_i(x)$ qui représente la composition des deux tables :

$$Ty_boxes_i^r = Ty_{[i\%Nb]} \circ T_i^r$$

3.7.5 Utilisation des tables

Maintenant que nos tables sont prêtes, il ne reste plus qu'à les utiliser dans la fonction Cipher. Pour faciliter la compréhension de l'algorithme, nous créons deux fonctions statiques `calculate_t_boxes` et `calculate_tyi_tables_and_xor_tables` qui, respectivement, cherchent les valeurs correspondantes de notre message dans les T-boxes (on applique donc AddRoundKey et SubBytes), puis on applique les tables Ty_i sur les nouvelles valeurs et on calcule l'opération XOR grâce aux tables XOR (on applique donc MixColumns). Le nouvel ordre d'appels des fonctions équivalent à la première implémentation devient (pour plus d'explications, voir 3.5.5) :

— 1. ShiftRows
— 2. AddRoundKey (ShiftRows sur la clé)
— 3. SubBytes
— 4. MixColumns

Nous pouvons représenter ces tours de calculs par un schéma tel que :

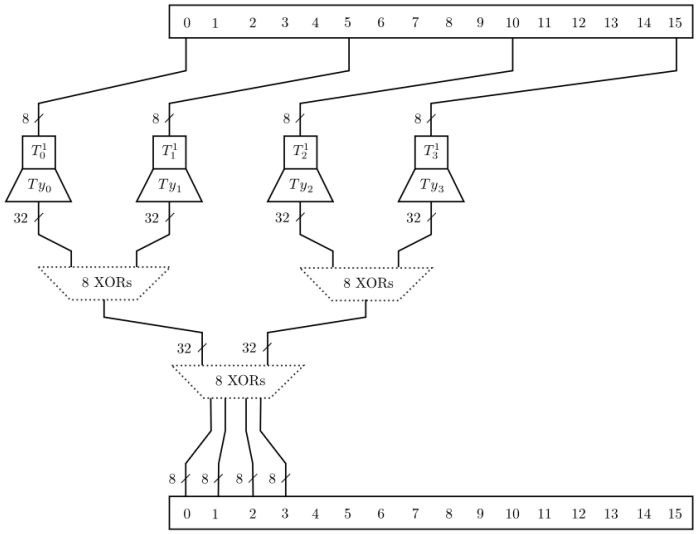

FIGURE 1 – Schéma simplifié de l'utilisation des tables [1]

Nous pouvons constater qu'il y a en tout 24 XORs au total que l'on peut représenter comme ceci :

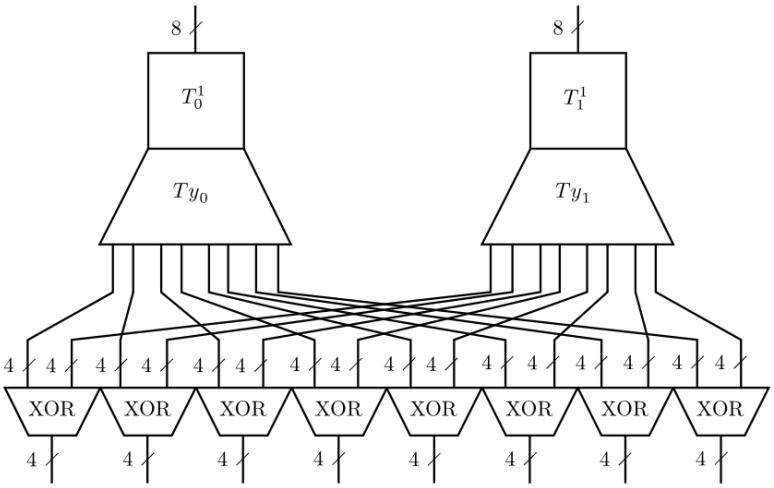

FIGURE 2 – Schéma des XORs [1]

Au dernier tour, on n'applique pas MixColumns. Une implémentation possible de la fonction Cipher est la suivante :

3.8 Compilation et génération des tables

Plusieurs fichiers sont nécessaires pour la compilation et le bon fonctionnement de notre stratégie. Notre plan serait de :

- Créer un fichier qui prend en argument la clé initiale et génère toutes les tables.
- Il écrit toutes les tables dans un fichier (par exemple tables.h).
- Compiler le fichier correspondant à la fonction Cipher (dans un autre fichier .c) et à sa fonction `main` (qui prend en argument un message clair à chiffrer). Ces deux fonctions sont dans un seul fichier à part de la génération des tables.

De ce fait, avec uniquement le deuxième exécutable, il n'est pas possible de voir la clé en clair dans le code. Représentons ce plan de compilation sur un schéma :

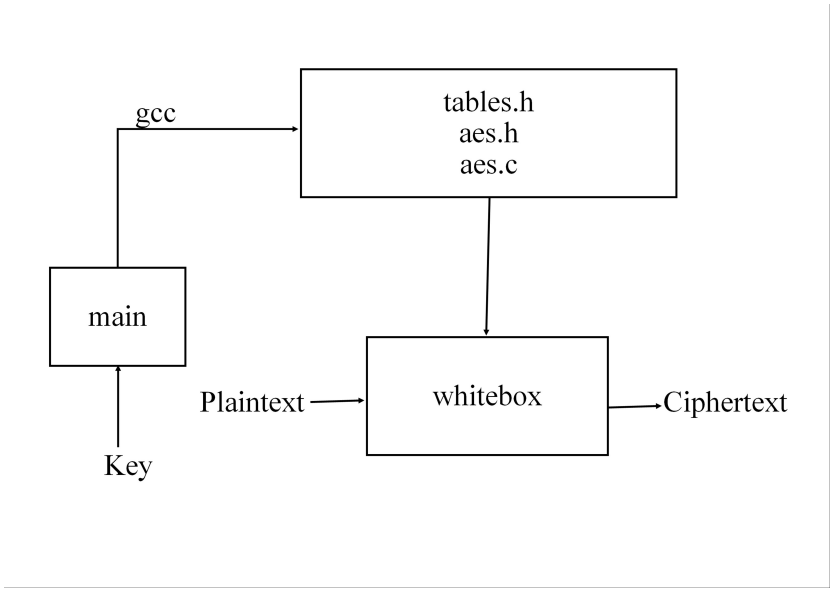

FIGURE 3 – Schéma de compilation

Ici, notre premier exécutable qui génère les tables et les écrit dans le fichier tables.h est main. Une fois le fichier généré, il compile aes.h, aes.c et tables.h ensemble pour former l'exécutable final whitebox. Celui-ci prend un message clair `m` en entrée et affiche un chiffré `c`. Cette stratégie permet de séparer la génération des tables et leurs utilisations dans le chiffrement pour éviter qu'un utilisateur puisse choisir sa propre clé ou alors de laisser la clé codée en clair dans le code.

4 Protection de l'implémentation

Bien que la clé soit "cachée" dans la T-boxes, il est possible pour un attaquant de trouver la clé très facilement en brute-force ou via une formule (voir section 5), si la table du tour 1 est connu de l'attaquant. Pour éviter cette attaque trés classique de se produire, nous réalisons de l'encodage/masquage pour rendre plus compliqué l'extraction de la clé.

4.1 Encodage/Masquage

4.1.1 Définition

Tout d'abord, donnons une définition de l'encodage. L'encodage est bijection appliquée aux entrées et aux sorties d'une table de substitution utilisée dans l'implémentation d'AES. Pour protéger une table T, on choisit deux fonctions bijectives f et g pour former une nouvelle table T' :

$$T' = g \circ T \circ f^{-1}$$

où f est l'encodage d'entrée et g l'encodage de sortie. L'encodage est appliqué à chaque octet de la table T. Plusieurs caractéristiques sont données à ces encodages :

- Le calcul est toujours correct : si $f(x)$ à l'entrée, puis on passe par T' et enfin on applique g^{-1}, on retrouve $T(x)$.
- Les encodages sont choisies aléatoirement.

Si on considère une deuxième table R' tel que :

$$R' = h \circ R \circ g^{-1}$$

où h est l'encodage de sortie et g l'encodage d'entrée. On peut remarquer que :

$$R' \circ T' = (h \circ R \circ g^{-1}) \circ (g \circ T \circ f^{-1}) = h \circ (R \circ T) \circ f^{-1}$$

Les encodages sont utilisés pour obscurcir le contenu de toutes les tables de correspondance (look-up tables) dans l'implémentation d'AES de Chow et al. Les sous-sections qui suivent montre un exemple de cet encodage sur des masques.

4.1.2 Masquage des tables Ty_i et des T-boxes

Pour renforcer la sécurité contre les attaques et empêcher un attaquant d'extraire des informations sensibles, nous générons des versions masquées des tables Ty_i et des T-Boxes, en utilisant un masque aléatoire. L'idée est qu'à chaque génération des tables, nous créons 10 masques aléatoires que nous stockons dans une table. Ensuite, dans la fonction `generate_t_boxes`, à chaque round, nous utilisons l'un de ces masques préalablement générés (correspondant au tour qui lui est attribué). Ainsi, toutes les tables sont efficacement masquées.

Dans la fonction `generate_tyi_tables`, où nous effectuons une multiplication, nous avons besoin des données non masquées pour éviter d'avoir des données faussées par le masquage. Pour cela, nous démasquons les données en effectuant un XOR avec le même masque utilisé à chaque round. Une fois le calcul effectué, nous remasquons le résultat avant de l'insérer dans les tables Ty_i. Enfin, nous sauvegardons le dernier masque utilisé, appelé `final_mask`, afin de l'utiliser dans la fonction `calculate_t_boxes` pour démasquer le résultat final du chiffrement.

```
void generate_random_mask(uint8_t mask[NB_ROUNDS]) {
  for (int i = 0; i < NB_ROUNDS; i++) {
    prng_init ((time (NULL) * getpid ()) % INT_MAX);
    mask[i] = rand() % 256;
    printf("0x%.2x\n",mask[i]);
  }
}

static void
generate_t_boxes (uint8_t round_key[Nb * (Nr + 1)][Nb])
{
  final_mask = mask[9];
  ...
  for (int x = 0; x < POSSIBLE_VALUES; x++)
    {
        t_boxes_data[r][i][x]
          = s_box[ x ^ block_get (key, i / Nb, i % Nb)] ^ mask[r];
    }
  ...
  // k9 et k10
 ...
  for (int x = 0; x < POSSIBLE_VALUES; x++)
    {
      t_boxes_data[NB_ROUNDS - 1][i][x]
          = s_box[ x ^ block_get (key_round_9, i / Nb, i % Nb)]
            ^ block_get (key_round_10, i / Nb, i % Nb) ^ mask[9];
    }
  ...

}

static void
generate_tyi_tables ()
{
  for (int r = 0; r < NB_ROUNDS - 1; r++)
  {
    for (int byte = 0; byte < TABLE_NUMBER; byte++)
    {
      for (int x = 0; x < POSSIBLE_VALUES; x++)
        {
          tyi_tables_data[r][byte][0][x ^ mask[r]]
              = (((gf_mul (2, x) << 24) | (x << 16) | (x << 8) | gf_mul (3, x))) ^
              ↪  mask[r+1];
          tyi_tables_data[r][byte][1][x ^ mask[r]]
              = ((gf_mul (3, x) << 24) | (gf_mul (2, x) << 16) | (x << 8) | x) ^
              ↪  mask[r+1];
          tyi_tables_data[r][byte][2][x ^ mask[r]]
              = ((x << 24) | (gf_mul (3, x) << 16) | (gf_mul (2, x) << 8) | x) ^
              ↪  mask[r+1];
          tyi_tables_data[r][byte][3][x ^ mask[r]]
              = ((x << 24) | (x << 16) | (gf_mul (3, x) << 8) | gf_mul (2, x)) ^
              ↪  mask[r+1];
        }
    }
  }
}
```

4.1.3 Masquage des tables composés

Il est aussi possible de protéger les tables composées par le biais de masques générés aléatoirement. Ici, nous appliquons un masque de la même manière que pour les tables T-boxes. Ici, il n'y a pas besoin de démasquer.

```
static void encoding_composed_tables(void) {
  for (int r = 0; r < NB_ROUNDS - 1; r++) {
    for (int i = 0; i < BLOCK_SIZE; i++) {
      for (int j = 0; j < BLOCK_SIZE; j++) {
        for (int x = 0; x < POSSIBLE_VALUES; x++) {
```

```
            ty_boxes_data[r][i][x] ^= mask[r];
          }
        }
      }
    }
}
```

Ces deux encodages nous permettent de mieux protéger nos tables contre les attaques. Cependant, nous sommes confrontés à un problème : lors de l'application des XOR, les masques sont immédiatement retirés. Ce comportement est normal car une règle de l'opération XOR est la suivante :

$$a \oplus a = 0$$

Pour pallier à ce souci, nous implémentons ce que l'on appelle les matrices de bijections.

4.2 Matrices de bijections

4.2.1 Nécessité de ces matrices

Dans une implémentation white-box de l'AES, l'utilisation des mixing bijections est essentielle pour éviter l'extraction de la clé via une analyse statistique. Le principe de base est la suivante : si les tables dépendantes de la clé, comme la table composée, sont protégées uniquement par des encodages d'entrée et de sortie de 4 bits, un attaquant peut exploiter les "signatures de fréquence" issues de fonctions comme $01 \cdot S_box, 02 \cdot S_box, 03 \cdot S_box$ (où $\cdot$ est le produit dans $GF(2^8)$) pour retrouver les encodages. En analysant les distributions de nibbles dans les sorties des tables, ces signatures peuvent être comparées à des signatures préconstruites et permettent de reconstruire les encodages, puis de remonter jusqu'à la valeur de la clé. L'ajout de mixing bijections, qui sont des transformations linéaires inversibles appliquées en entrée et sortie de ces tables, brouille ces distributions et rend cette méthode d'analyse inefficace. Elles jouent ainsi un rôle central dans la sécurité de l'implémentation white-box en assurant une meilleure diffusion de l'information (pour plus d'informations sur cette attaque, voir l'article [1])

4.2.2 Définition et principe

Définissons tout d'abord la transformation linéaire. une transformation linéaire est une fonction T qui prend un vecteur et le transforme en un autre vecteur de manière à préserver l'addition des vecteurs et la multiplication par un scalaire. Donc ses proprétés sont :

— Additivité : Pour deux vecteurs u et v, la transformation satisfait :

$$T(u + v) = T(u) + T(v)$$

— Homogénéité : Pour un scalaire a et un vecteur u, la transformation satisfait :

$$T(a \cdot u) = a \cdot T(u)$$

Les matrices de bijections sont des transformations linéaires représentées en matrices inversibles. Elles améliorent la diffusion des bits dans les look-up tables, ce qui rend plus difficile l'analyse des dépendances entre les entrées et les sorties. Rappelons tout d'abord la définition de la table composée :

$$Ty_boxes_i^r = Ty_{[i\%Nb]} \circ T_i^r$$

Ces matrices de bijections sont construites de la façon suivante :

- Tout d'abord, on initialise deux matrices de 8 bits et 32 bits respectivement de taille 10×16 et 10×4 (la première matrice représente 16 entiers de 8 bits, tandis que la deuxième matrice représente 4 entiers de 32 bits). On appellera les deux matrices respectivement ML et MB.
- Ensuite, on remplit ces deux matrices avec des valeurs aléatoires dans $GF(2)$ et on vérifie si ces matrices sont inversibles.
- Si une des deux matrices ne sont pas inversibles, on recommence l'opération.
- Pour MB, il y a $32 \times 32 = 1024$ bits, donc 2^{1024} matrices possibles. Le nombre de matrices inversibles dans $GF(2)$ est

  $$(2^{32} - 1)(2^{32} - 2) \cdots (2^{32} - 2^{31})$$

  car chaque ligne doivent être linéairement indépendante les uns des autres. La probabilité que la matrice soit inversible est :

  $$\frac{(2^{32} - 1)(2^{32} - 2) \cdots (2^{32} - 2^{31})}{2^{32 \cdot 32}} = \prod_{i=0}^{31} \left(1 - 2^{i-32}\right) \approx 0.288.$$

Pour commencer, sélectionnons toutes les bijections nécessaires :

- Pour chaque tour de 2 à 10, on sélectionne 16 mixing bijections (correspondant à chaque position d'un octet du message) de 8 bits → 8 bits (soit 144 mixing bijections au total). Celles-ci seront composées à l'entrée de chaque T-box lors des tours 2 à 10. On ne doit pas appliquer un encodage d'entrée au premier tour car le message d'entrée étant en clair, les données risquent d'être faussées par l'encodage d'entrée.
- Pour chaque tour de 1 à 9, on sélectionne 4 mixing bijections (correspondant à chaque colonne du résultat de la multiplication de la matrice MC par une sortie x) de 32 bits → 32 bits (soit une mixing bijection pour chacune des quatre étapes de multiplication matricielle de chacun de ces tours). Celles-ci seront composées à la sortie de chaque table Ty_i lors des tours 1 à 9. Cet encodage se calcule par la transformation linéaire MB. Ici, on ne doit pas encoder à la sortie de la table composée car nous voulons que les données sortent en chiffrées et non encodées.

Maintenant que nous avons nos matrices, nous pouvons les utiliser pour encoder nos tables composées. Nous allons donc appliquer la transformation suivante :

$$Ty_boxes_i^r(x) = MB \circ Ty_{[i\%Nb]} \circ T_i^r \circ L_i^{r^{-1}} \text{ pour i=0...15 et r=2...9}$$

où MB est la matrice de 32 bits, ty_i la table Ty_i, T_i^r la T-box et $L_i^{r^{-1}}$ est une tranformation linéaire inverse définie comme par exemple.

$$L_i^r(x) = ML^{-1}[r][i] * x$$

Où ML^{-1} est la matrice inverse de ML. La construction de la table composée pour le premier et dernier tour est donc la suivante :

$$Ty_boxes_i^r(x) = MB \circ Ty_{[i\%Nb]} \circ T_i^r \text{ pour i=0...15 et r=1}$$
$$Ty_boxes_i^r(x) = Ty_{[i\%Nb]} \circ T_i^r \circ L_i^{r^{-1}} \text{ pour i=0...15 et r=10}$$

Un code pour la construction de la table composée serait :

```
for (int r = 0; r < NB_ROUNDS; r++) {
  for (int i = 0; i < BLOCK_SIZE; i++) {
    for (int x = 0; x < POSSIBLE_VALUES; x++) {
      uint8_t tmp = x;
      if (r > 0) {
        // .inv pour inverser la matrice
        tmp = ML[r-1][i].inv * x;
      }
      // Construction classique de la table composée, stocké dans tmp
      tmp = s_box[tmp ^ block_get (key, i / Nb, i % Nb)];
      if (r < NB_ROUNDS - 1) {
        Ty_boxes[i][r][x] = tmp;
      }
      Ty_boxes[i][r][x] = MB[r][i % Nb] * tmp;
    }
  }
}
```

Les sorties de ces tables sont indexées par la suite dans les tables XOR. Cependant, à la fin de la troisième étape des tables XOR, la valeur 32 bits résultante est maintenant (où MC est la matrice de MixColumns) :

$$MB \circ MC[z_0, z_1, z_2, z_3]^T$$

Nous devons supprimer la transformation MB et appliquer également les mixing bijections de 8 bits requises pour le tour suivant pour que les nouvelles entrées soient "propres" et que le prochain tour fonctionne correctement. On décompose MB en quatre nouvelles tables de 8 bits $\rightarrow$ 32 bits sont introduites pour supprimer l'effet de MB. Ces tables sont générées à l'aide de la technique courante de décomposition d'une multiplication matricielle en un OU exclusif de quatre vecteurs de 32 bits :

$$MB^{-1}\begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \end{bmatrix} = MB_0^{-1}\begin{bmatrix} z_0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \oplus MB_1^{-1}\begin{bmatrix} 0 \\ z_1 \\ 0 \\ 0 \end{bmatrix} \oplus MB_2^{-1}\begin{bmatrix} 0 \\ 0 \\ z_2 \\ 0 \end{bmatrix} \oplus MB_3^{-1}\begin{bmatrix} 0 \\ 0 \\ 0 \\ z_3 \end{bmatrix}$$

Ici, MB_0^{-1}, MB_1^{-1}, MB_2^{-1} et MB_3^{-1} sont les nouvelles tables correspondants aux termes de la somme de droite. Chacune de ces nouvelles tables prend en entrée un entier de 8 bits et renvoie un entier de 32 bits.

Aprés avoir supprimé l'effet de la bijection linéaire MB, on veut appliquer une nouvelle bijection linéaire L_3 sur 32 bits utilisé pour encoder les quatre octets de sorties avant le round 3. On doit prendre

en compte l'effet de ShiftRows, car AES permute les positions des octets à chaque round. On la construit en combinant quatre bijections 8 bits $\rightarrow$ 32 bits.

$$L^3 = L_0^3 \parallel L_1^3 \parallel L_2^3 \parallel L_3^3$$

Ici, L_i^3 est une bijection agissant sur l'octet à la position i. Les quatre octets qui vont arriver sur la première colonne aprés ShiftRows sont les octets en position : 0, 13, 10 et 7.

Représentons le résultat sur un schéma :

$$\begin{bmatrix} s_0 & s_1 & s_2 & s_3 \\ s_4 & s_5 & s_6 & s_7 \\ s_8 & s_9 & s_{10} & s_{11} \\ s_{12} & s_{13} & s_{14} & s_{15} \end{bmatrix} \rightarrow \begin{bmatrix} s_0 & s_1 & s_2 & s_3 \\ s_7 & s_4 & s_5 & s_6 \\ s_{10} & s_{11} & s_8 & s_9 \\ s_{13} & s_{14} & s_{15} & s_{12} \end{bmatrix}$$

Pour préparer le bon encodage de ces 4 octets, on doit appliquer :

$$L^i = L_0^i \parallel L_7^i \parallel L_{10}^i \parallel L_{13}^i \quad \text{pour } i = 3 \ldots 9$$

Cette nouvelle bijection linéaire permet le bon positionnement des octets dans la matrice d'état et d'éviter que les données sont faussées. Nous pouvons créer désormais une nouvelle table MixBijOut tel que :

$$MixBijOut_i^r = L^r \circ MB_i^{-1}$$

Avec toutes ces modifications, nous pouvons désormais l'implémenter de la façon suivante :

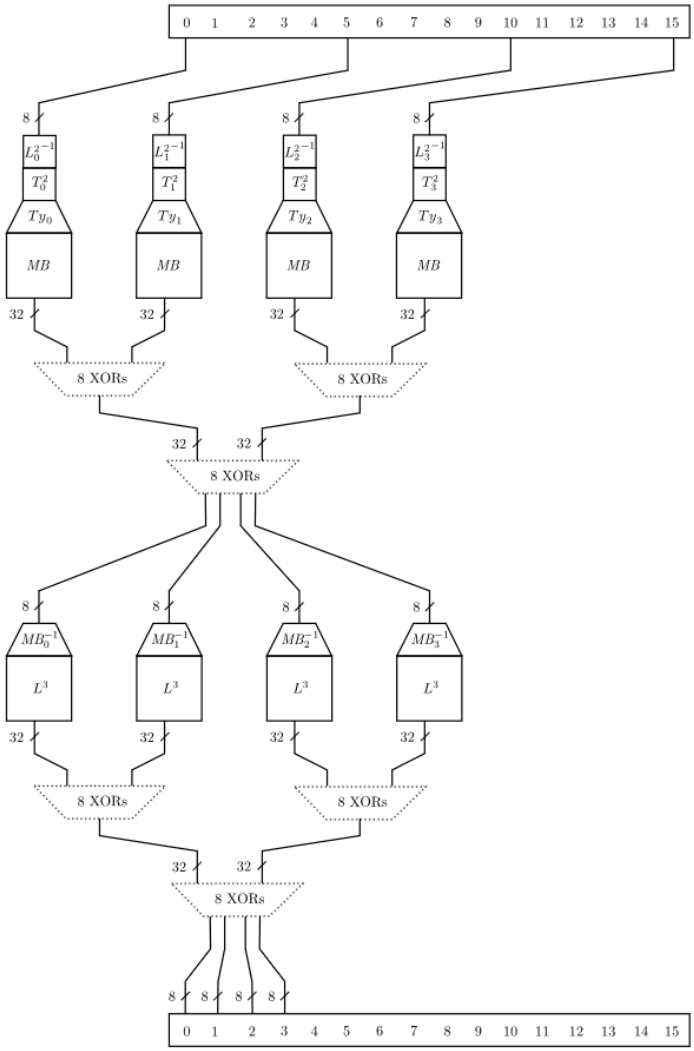

FIGURE 4 – Schéma de l'utilisation des tables avec l'encodage et les mixing bijections pendant le tour 2

5 Attaque BGE

L'attaque BGE, proposée par Billet, Gilbert et Ech-Chatbi [3], est une attaque algébrique puissante visant certaines implémentations du chiffrement AES conçues pour cacher la clé, même lorsqu'un attaquant peut observer ou manipuler toutes les opérations internes. L'objectif de cette attaque est de reconstruire une version isomorphe du chiffrement AES à partir de la structure des tables utilisées dans l'implémentation, puis d'en extraire la clé secrète.

5.1 Structure des tables composées

Comme dit précédemment, les tables composées sont construites à partir de la composition des T-boxes et des tables Ty_i et de deux transformations linéaires. Nous rappelons ici la définition de la table composée :

$$Ty_boxes_i^r(x) = MB \circ ty_i \circ T_i^r \circ L_i^{r^{-1}} \quad \text{pour } i = 0 \ldots 15 \text{ et } r = 2 \ldots 9$$

5.2 Principe de l'attaque

Pour cette attaque, nous notons P_i la combinaison de l'encodage d'entrée et des mixing bijections, et Q_i la combinaison de l'encodage de sortie et des mixing bijections. Nous avons donc :

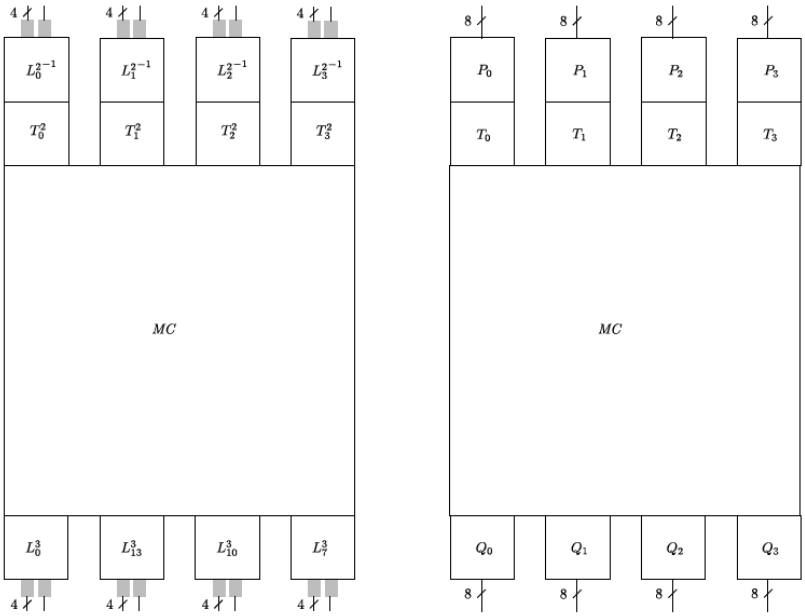

FIGURE 5 – Schéma de la notation de l'attaque

Notons x_0, x_1, x_2 et x_3 les valeurs d'entrée et y_0, y_1, y_2 et y_3 les valeurs de sortie tels que :

$$\begin{aligned}
y_0 &= Q_0(02 * T_0'(x_0) \oplus 03 * T_1'(x_1) \oplus 01 * T_2'(x_2) \oplus 01 * T_3'(x_3)) \\
y_1 &= Q_1(01 * T_0'(x_0) \oplus 02 * T_1'(x_1) \oplus 03 * T_2'(x_2) \oplus 01 * T_3'(x_3)) \\
y_2 &= Q_2(01 * T_0'(x_0) \oplus 01 * T_1'(x_1) \oplus 02 * T_2'(x_2) \oplus 03 * T_3'(x_3)) \\
y_3 &= Q_3(03 * T_0'(x_0) \oplus 01 * T_1'(x_1) \oplus 01 * T_2'(x_2) \oplus 02 * T_3'(x_3))
\end{aligned}$$

Ces équations sont obtenues suite à la définition de la matrice MC, représentant l'opération de la fonction MixColumns. Q_i fait donc référence à l'opération MixColumns sur la ligne i. Q_i fait fuiter des informations sur ces quatre identités car elles combinent des bijections de sortie et des transformations linéaires. En

effet, elles ne masquent pas assez les relations algébriques internes même si elles combinent des encodages et des transformations linéaires pour embrouiller les données. Nous pouvons donc construire une approximation qui s'en rapproche par une transformation affine inconnue (à partir d'observations sur les entrées/-sorties des tables de l'implémentation). On note cette approximation $\widetilde{Q}_i$ et on la définit comme suit :

$$\widetilde{Q}_i = Q_i \circ A_i$$

Où A_i est une transformation affine inconnue suivie d'un XOR avec une constante. Ces approximations sont construites en analysant un nouvel ensemble de lookup tables. y_0 peut être vu comme une fonction des x_0, x_1, x_2 et x_3 (représentant les sorties de T') tel que :

$$y_0 = f(x_0, x_1, x_2, x_3)$$

La valeur y_0 dépend de x_0, x_1, x_2 et x_3. Mais si x_2 et x_3 sont fixés comme une constante, alors y_0 ne dépend que de x_0 et x_1. Si on fixe x_2 et x_3 à 00, on peut construire de nouvelles lookup tables pour $y_0 = f_{00}(x_0)$ et $y_0 = f_{01}(x_0)$. À partir de la première équation, on obtient :

$$f_{00}(x_0) = 02 * T'_0(x_0) \oplus \beta_{00}$$
$$f_{01}(x_0) = 02 * T'_0(x_0) \oplus \beta_{01}$$

où β_{00} et β_{01} sont des constantes inconnues de 8 bits. À partir de ces fonctions, nous pouvons construire de nouvelles lookup tables. On note ces fonctions comme ceci :

$$f_{00} = Q_0 \circ \oplus\beta_{00} \circ 02 * T'_0 f_{01} = Q_0 \circ \oplus\beta_{01} \circ 02 * T'_0$$

On remarque que :

$$f_{01} \circ f_{00}^{-1} = Q_0 \circ \oplus\beta \circ Q_0^{-1}$$

où $\beta = \beta_{01} \oplus \beta_{00}$. Il y a exactement 256 bijections possibles de cette forme, car β est un entier sur 8 bits. Cet ensemble forme un groupe abélien $(G, \circ)$. On peut en extraire une base de 8 éléments $\{g_0, ..., g_7\}$, et construire un isomorphisme vers $\mathbb{F}_2^8$:

On utilise f_{00}^{-1} pour "ramener" tous les éléments vers une forme canonique. A partir de ce groupe, on note 8 éléments, $(g_0, g_1, ..., g_8)$ comme une base de G. Il a 8 éléments car le groupe G est abélien de taille 256. Il est isomorphe à l'espace vectoriel $\mathbb{F}_2^8$ donc il est de dimension 8. Par conséquent, il posséde ces 8 éléments tels qu'un sous-ensemble puisse être composé pour générer n'importe quel élément de groupe. Avec ces propriétés, on construit un isomorphisme :

$$\phi : (G, \circ) \to (\mathbb{F}_{2^8}, \oplus)$$

Ce qui permet de reconstruire une approximation $\widetilde{Q}_0$ par :

$$\widetilde{Q}_0(\phi(g)) = g(00)$$

Avec le même processus, on construit $\widetilde{Q}_1$, $\widetilde{Q}_2$ et $\widetilde{Q}_3$. Ainsi, les encodages de sortie peuvent être inversés efficacement :

$$\widetilde{Q}_i^{-1} \circ Q_i = A_i^{-1}$$

Chaque Q_i est alors exprimable comme :

$$Q_i(x) = M_i(x) \oplus q_i$$

avec M_i une matrice inversible et q_i un vecteur constant.

Une fois les approximations construites et les encodages inversés, deux méthodes principales permettent d'extraire les octets de la clé :

— **Méthode par force brute** : Grâce aux fonctions P_i et Q_i désormais connues ou approximées, l'attaquant peut simuler le comportement interne des transformations AES, notamment les entrées/sorties des T-boxes. Pour chaque octet de la clé candidate k_i (par exemple dans le round final), il est possible de :

1. Appliquer la transformation inverse P_i^{-1} aux données observées pour "remonter" jusqu'à la valeur claire.
2. Deviner une valeur de k_i et appliquer $S(k_i \oplus m_i)$.
3. Comparer le résultat avec la sortie réelle de la table Ty_i.

En testant ainsi les 256 valeurs possibles pour chaque octet, l'attaquant identifie la valeur correcte lorsque le comportement observé coïncide avec celui attendu. Ce processus est itératif et peut être appliqué à chaque octet indépendamment, rendant l'extraction efficace une fois les approximations bien établies.

— **Méthode algébrique directe** : Une approche plus rapide repose sur l'analyse des transformations dans les tables elles-mêmes. Une relation algébrique directe permet de retrouver un octet de clé à partir de la table composée, sans nécessiter de tests exhaustifs :

$$a_i = S_box^{-1} \circ ty_{[i\%Nb]}^{-1} \circ (Ty_{[i\%Nb]} \circ T_i^1)(0)$$

Cette équation exploite le fait que, pour une entrée nulle (0), les transformations linéaires et bijections révèlent une combinaison dépendant uniquement de la clé. En inversant soigneusement les étapes, on "remonte" à la valeur de l'octet de clé ayant été injecté dans le round. Cette méthode est extrêmement rapide une fois les bijections ty_i inversées, car elle contourne toute phase de test et se base sur une structure théorique rigoureusement extraite de l'implémentation.

L'étape la plus coûteuse est la construction des approximations des codages, estimée à 2^{24} étapes par approximation, soit 2^{28} pour un tour complet d'AES.

5.3 Application sur notre implémentation

Sur notre implémentation, il est possible de réaliser une attaque pour trouver les masques et donc extraire la clé. Notre stratégie est la suivante :

— Connaissant le dernier masque, on doit dans un premier temps retrouver les masques du tour 9 à 0 (le dernier masque correspondant au tour 10).

— Démasquer la T-box au premier tour avec $mask_1$, puis appliquer la table de substitution S_box inversé pour retrouver un octet de la clé à chaque position $0 \leq i \leq 15$.
— Effectuer InvShiftRows pour retrouver la clé initiale (car la clé initiale a été modifiée par ShiftRows).

Dans un premier temps, nous devons, connaissant le dernier masque, remonter jusqu'au premier masque grâce à une équation tel que :

$$tyi_tables[r][0][i\%Nb][x \oplus mask[r]] = AES_MIXCOLUMNS_{[i\%Nb]}(x) \oplus mask_{r+1}$$

où :
— $AES_MIXCOLUMNS_{[i\%Nb]}(x)$ est définit comme :

$$(gf_mul(2, x) << 24)|(x << 16)|(x << 8)|gf_mul(3, x)$$

— $mask_{r+1}$ est connu pour le tour r (initialement on connait $mask_{10}$).
— `tyi_tables` est notre table Ty_i correspondante (voir section 3.7.2).

Nous cherchons à retrouver $mask_r$ en bruteforçant tous les $x \in [0, 256]$ et en vérifiant si l'on retrouve la valeur `expected` dans la table. Une fois un match trouvé pour une entrée i, on déduit :

$$mask_r = x \oplus i$$

Cette relation vient directement de la relation structurelle utilisée dans l'AES pour construire les tables Ty_i masquées :

$$tyi_tables[r][0][i\%Nb][x \oplus mask_r] = AES_MIXCOLUMNS_{[i\%Nb]}(x) \oplus mask_{r+1}$$

On répete l'opération pour $8 \geq r \geq 0$, ce qui permet de retrouver tous les masques internes utilisés. Nous pouvons implémenter une fonction comme suit :

```
bool found = false;
uint8_t mask0;
void extract_mask(void) {
  // final_mask connu
  uint8_t mask_r_1 = final_mask;
  uint8_t mask_r;
  for (int r = 8; r >= 0; r--) {
    found = false;
    for (uint8_t x = 0; x < 256; x++) {
      uint32_t expected =
          (gf_mul(2, x) << 24) | (x << 16) | (x << 8) | gf_mul(3, x);
      expected ^= mask_r_1;
      for (uint16_t i = 0; i < 256; i++) {
        if (tyi_tables[r][0][0][i] == expected) {
          mask_r = x ^ i;
          found = true;
          break;
        }
      }
      if (found) {
        break;
      }
    }
```

```
    mask_r_1 = mask_r;
  }
  mask0 = mask_r;
  return NULL;
}
```

Une fois le masque $mask_r$ obtenu, nous pouvons l'utiliser pour retrouver les octets de la clé AES initiale à partir de la T-boxes (`t_boxes` pour notre implémentation), construite comme suit :

$$T_i^1(x) = S_box[x \oplus \hat{k_0}[i]] \oplus mask_1$$

En fixant $x = 0$, cette relation devient :

$$T_i^1(0) = S_box[\hat{k_0}[i]] \oplus mask_1$$

Cela permet, par inversion de la S_box, de retrouver chaque octet de la clé :

$$\hat{k_0}[i] = S_box^{-1}[T_i^0(0) \oplus mask_1]$$

La clé extraite est ensuite soumise à l'opération InvShiftRows, qui corrige le décalage introduit par la tranformation ShiftRows de l'AES. Cela permet de retrouver la clé initiale correcte. On peut implémenter ces opérations comme ceci :

```
void extract_key(block_t *key) {
  for (int i = 0; i < 16; i++) {
    uint8_t sbox_val = t_boxes[0][i][0] ^ mask0;
    uint8_t key_byte = inv_s_box[sbox_val];
    block_set(key, key_byte, i / 4, i % 4);
  }
  inverse_shift_rows(key);
}
```

Grâce à ce type de stratégie, il est possible de contourner les protections de masquage présentes dans l'implémentation Whitebox et de récupérer la clé AES utilisée.

6 Conclusion

L'algorithme AES, aujourd'hui largement adopté comme standard de chiffrement symétrique, repose sur une structure rigoureuse mêlant substitution, permutation et opérations linéaires. Cette conception en plusieurs tours garantit à la fois la diffusion et la confusion nécessaires pour une bonne sécurité cryptographique. En particulier, l'agencement des opérations comme `SubBytes`, `ShiftRows`, `MixColumns` et `AddRoundKey` peut être optimisé : certaines de ces opérations peuvent être permutées ou fusionnées, notamment pour permettre une implémentation efficace via des tables de correspondance, appelées *lookup tables*.

Ces tables pré-calculées, comme les T-boxes ou les tables Ty_i, offrent une amélioration significative en termes de performance, en réduisant le nombre d'opérations nécessaires à l'exécution du chiffrement ou du déchiffrement. En

substituant des calculs dynamiques par des accès mémoire, elles permettent d'accélérer considérablement le traitement, ce qui est crucial pour des applications en temps réel ou embarquées.

Cependant, cette optimisation introduit aussi de nouveaux vecteurs d'attaque, notamment dans le contexte du chiffrement en environnement whitebox, où l'adversaire a accès non seulement aux entrées et sorties, mais aussi à l'implémentation elle-même. Pour contrer cela, plusieurs stratégies de protection ont été développées, comme l'encodage des tables ou l'utilisation de transformations aléatoires internes. Ces contre-mesures visent à masquer les opérations intermédiaires et à empêcher l'extraction directe de la clé ou des données sensibles.

Malgré ces efforts, certaines attaques ciblées ont vu le jour, dont l'attaque BGE (Billet-Gilbert-Ech-Chatbi), qui illustre la complexité du défi de la sécurité whitebox. Cette attaque, même si elle nécessite des ressources non triviales, démontre que la simple obfuscation ne suffit pas toujours à garantir une sécurité robuste lorsque l'attaquant a un accès complet au programme. Elle met en lumière les limites actuelles des implémentations whitebox et souligne la nécessité de revoir certaines hypothèses de sécurité dans ces modèles.

En somme, la construction d'AES et ses variantes optimisées comme Whitebox AES restent des sujets riches d'analyse et d'innovation. Si des avancées considérables ont été faites, tant sur le plan théorique que pratique, les attaques comme BGE rappellent que la cryptographie est un champ en constante évolution, où la sécurité d'aujourd'hui peut être remise en question demain.

Une question demeure alors ouverte : comment les futurs algorithmes de chiffrement évolueront-ils pour répondre aux exigences croissantes de sécurité face à des attaques de plus en plsus sophistiquées ? Seront-ils capables de résister durablement à l'ingéniosité des cryptanalystes ?

Références

[1] J.A. Muir. A tutorial on white-box aes. `https://eprint.iacr.org/2013/104.pdf`, 2013.

[2] National Institute of Standards and Technology. Specification for the advanced encryption standard (aes). `https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf`, 2001.

[3] Henri Gilbert Olivier Billet and Charaf Ech-Chatbi. Cryptanalysis of a white box aes implementation. `https://www.researchgate.net/publication/225241894_Cryptanalysis_of_a_White_Box_AES_Implementation`, 2005.